



Adaptación de Skills IA al Ecosistema BCV/BACC

Presentación Ejecutiva

Proyecto: skills de agente IA para los microservicios backend de Apertura de Cuentas Comerciales (BACC)

Ecosistema: BCV/BACC · Interbank · Entregable: NTT DATA

Cronograma: 13 ago → 09 oct 2026 · Sprint 2 en curso (4 sprints de 2 semanas + cierre) · Pipeline HU → DHU → código con graphify (~85 % menos tokens)

01 · Contexto del Proyecto

Iniciativa, problema, metodología y valor cuantificado

¿Qué es este proyecto?

1

Ecosistema BCV/BACC

7 microservicios backend Java/Spring Boot fuertemente integrados: Azure Service Bus, OpenFeign, SQL Server + Cosmos DB, Key Vault.

2

Qué construimos

Skills de agente IA (instrucciones especializadas para asistentes de código tipo GitHub Copilot) para generar, mantener y diagnosticar los microservicios.

3

Flujo objetivo

Convertir una Historia de Usuario (HU) de negocio en una Historia Técnica (DHU) detallada y, opcionalmente, en cambios de código.

Problema y objetivo principal

Problema a resolver

Analizar e implementar Historias de Usuario con un LLM que lee repositorios completos es caro y lento (~50 000–90 000 tokens por HU) y no sigue un estándar corporativo. Dificulta escalar el uso de asistentes de IA en el desarrollo de BACC.

Objetivo principal

Construir skills de agente que codifiquen el conocimiento del dominio BACC e implementen el pipeline HU → DHU → código, usando graphify como motor de análisis local para reducir el consumo de tokens ~85 %, alineados al template corporativo ibk-hu-technical-refinement.

Estado actual → estado objetivo

Dimensión	Estado actual	Estado objetivo
Análisis de HUs	LLM lee repos completos (~70k tokens/HU)	graphify local + contexto mínimo (~10k tokens/HU)
Especificación	Informal, sin estándar	DHU con template ibk-hu-technical-refinement
Implementación	Manual, sin guía de dominio	Asistida por skills especializados + ramas feature/HU-...
Conocimiento de dominio	Implícito en el equipo	Codificado en 13 skills reutilizables

Valor para el banco (cuantificado)

~85 % menos tokens

Por HU analizada: de ~70 000 a ~10 000 tokens.

Onboarding acelerado

Equipo de desarrollo autónomo en el uso de asistentes de IA.

13 skills de dominio

Capturan convenciones BACC: hexagonal, ASB, OpenFeign, JPA, Cosmos, observabilidad y testing.

Trazabilidad corporativa

DHU estándar + contexto versionado (service-map.md, gotchas.md).

Pipeline reproducible

HU → DHU → código con validación y gates propios (DoR / DoD).

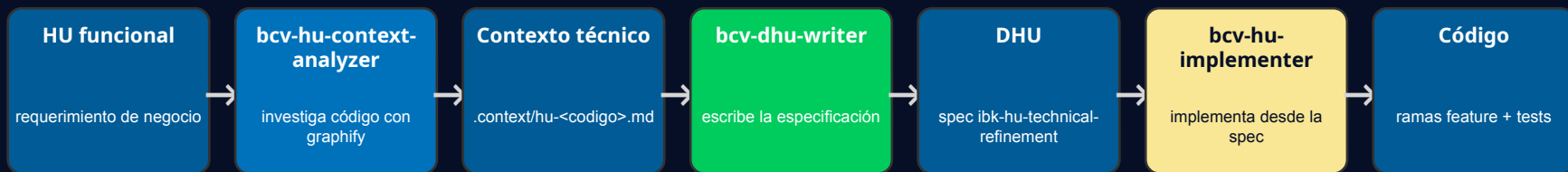
Metodología: SDD + BMAD

Spec-Driven Development — la especificación (DHU) es el artefacto central y va antes del código · BMAD: Understand → Design → Build → Validate

Fase SDD	Acción del flujo BCV
Especificar (entrada)	HU funcional de negocio
Research-Driven Context	bcv-hu-context-analyzer investiga el código real con graphify → .context/hu-<codigo>.md
Especificar formalmente	bcv-dhu-writer escribe la DHU (criterios de aceptación, endpoints, mapa técnico)
Implementar desde la spec	bcv-hu-implementer aplica la DHU en ramas feature (dry-run / apply)
Verificar / feedback	Linter + tests, revisión humana y gotchas que vuelven al contexto

Flujo visual: cómo SDD se aplica en BCV

La especificación (DHU) va antes del código; el research usa graphify (código real, local, ~0 tokens de LLM) en vez de que el LLM lea repositorios completos.



Verificar / feedback

Feedback (linter + tests + gotchas) retroalimenta a bcv-hu-context-analyzer para la siguiente HU.

¿Por qué no Spec-Kit ni OpenSpec?

Se evaluaron los frameworks SDD genéricos y se descartaron a favor del flujo custom BCV:

Aspecto	Flujo BCV (custom)	Spec-Kit / OpenSpec
Formato de spec	DHU (ibk-hu-technical-refinement)	spec.md propio / specs/ + deltas
Investigación de código	graphify — grafo local, ~0 tokens	LLM lee archivos
Contexto multi-servicio	workspace + service-map.md + gotchas.md	repo-local
Alineación corporativa	Alta (BCV/BACC)	Baja (genérico)
Eficiencia de tokens	Contexto mínimo y consultas quirúrgicas	Mayor consumo por HU
Control del pipeline	Gates propios, sin commit automático	Flujo genérico

Ahorro de tokens

Escenario	Tokens estimados por HU
Con skills + graphify	~8 000 – 12 000
Con skills, sin graphify	~25 000 – 45 000
Sin skills ni graphify	~50 000 – 90 000

Ahorro vs. LLM puro: ~85 % menos

HU de ejemplo «Agregar oficina registral en canal BCW»: ~10 000 tokens con skills + graphify, frente a ~33 000 sin graphify y ~70 000 sin skills ni graphify.

Stack tecnológico

Capa	Tecnología
Lenguaje	Java 21 (Java 17 en service-point-service)
Framework	Spring Boot 3.x (ads-spring-boot-dependencies)
Arquitectura	Hexagonal / Ports & Adapters (6 de 7 servicios)
Mensajería	Azure Service Bus (ads-spring-boot-starter-messaging)
Persistencia	SQL Server (JPA/Hibernate), Azure Cosmos DB
Clientes HTTP	OpenFeign
Seguridad	JWT Bearer + Azure Key Vault + Spring Cloud Config
Observabilidad	bcv-commons-observability, Spring Actuator

02 · Alcance del Proyecto

Qué incluye y qué queda fuera

Resumen de alcance

7

microservicios backend

Java / Spring Boot · BACC

13

skills de agente IA

3 pipeline + 10 implementación

~85 %

menos tokens por HU

de ~70k a ~10k tokens

8 sem.

de duración

13 ago → 09 oct 2026

Incluido — 7 microservicios backend BACC

Servicio	Arquitectura
party-lifecycle-management-service	Hexagonal (orquestador central)
channel-activity-service	Hexagonal (procesador stateless)
compliance-service	Hexagonal
current-account-service	Hexagonal
customer-service	Hexagonal
account-opening-reporting-service	Hexagonal (SQL + Cosmos)
service-point-service	Layered (legacy)

Incluido — 13 skills de agente

Pipeline (3)

bcv-hu-context-analyzer

bcv-dhu-writer

bcv-hu-implementer

Implementación (10)

bcv-hexagonal-architecture · bcv-clean-architecture · bcv-java-spring-boot · bcv-openapi-design · bcv-openfeign · bcv-azure-service-bus · bcv-spring-data-jpa-sql-server · bcv-cosmos-db · bcv-commons-observability · bcv-testing

Flujo completo HU → DHU → implementación

Con graphify y contexto mínimo versionado por servicio: service-map.md · architecture-conventions.md · cross-service-patterns.md · gotchas.md

Fuera de alcance / Etapa 2 candidata

Ítem	Motivo
Frontend, pantallas y lógica de presentación	El flujo analiza microservicios backend únicamente
Migración de service-point-service a hexagonal	Deuda técnica detectada — candidata a trabajo futuro
Completar rest-api.yaml vacíos en varios servicios	Deuda de documentación de contratos
Activar quality gates (SpotBugs / PMD en skip=true)	Mejora de calidad independiente
Otros ecosistemas no BACC	Fuera del dominio del proyecto

03 · Equipo de Trabajo

Equipo NTT DATA, contraparte del banco y responsables por entregable

Equipo IA (NTT Data)

Jose Luis Mejia Rojas

Arquitecto IA

Disponibilidad: 13 ago – 10 sep 2026

Arquitectura del flujo HU → DHU, decisiones de contexto y adopción de graphify, diseño del workspace, validación de DHUs e interlocución con el cliente.

Oscar Fabian Castro Severino

Ingeniero IA

Disponibilidad: 13 ago – 09 oct 2026

Implementación y refinamiento de skills, grafos graphify, ejecución del pipeline (análisis, DHU, implementación), pruebas y reportes. Desde el 10 sep: soporte y afinaciones.

Equipo de desarrollo del banco (colaboración)

Contraparte que valida en la práctica: dispone del entorno en su propia PC y ejecuta iteraciones con feedback sobre los skills.

1

Sergio Manuel

Líder técnico

2

Lionel Gonzales

Desarrollador

3

Fernando Camargo

Desarrollador

4

Jose Peramas

Desarrollador

Responsable por entregable

Entregable	Responsable
Arquitectura del flujo + workspace y convenciones	Jose Luis (Arquitecto IA)
Contexto versionado (service-map.md, convenciones)	Jose Luis
Skills del pipeline (3)	Jose Luis + Oscar
Skills de implementación (10)	Oscar
Grafos graphify (7 repos)	Oscar
Análisis de HUs y generación de DHU	Oscar (ejecución) + Jose Luis (validación)
Implementación y reporte	Oscar
gotchas.md por servicio	Jose Luis y Oscar (con soporte IA)
Validación y feedback	Equipo dev banco (Sergio Manuel, líder técnico)

Riesgo de dotación

10 sep 2026 — Jose Luis finaliza su participación (mitad del proyecto)

Su trabajo queda culminado y traspasado en esa fecha. A partir de ahí, Oscar queda como único recurso del equipo IA, en rol de soporte y afinaciones finales.

Mitigación

Traspaso planificado con trabajo culminado antes del 10 sep + cobertura de Oscar en soporte y afinaciones.

04 · Roadmap del Proyecto

4 sprints de 2 semanas + cierre · SDD + BMAD · 13 ago → 09 oct 2026

Esfuerzo por bloque (estimación)

Bloque de trabajo	Sprints	Esfuerzo
Consolidación de repos + entorno	S1	10 %
Skills del pipeline + graphify	S1	20 %
Skills de implementación + iteración	S2–S3	40 %
Validación con equipo banco + correcciones	S4	15 %
Documentación, onboarding y cierre	S4 + cierre	10 %
Soporte y afinaciones (Oscar)	Cierre	5 %

Equipo de 2 personas durante ~8 semanas (4 sprints de 2 semanas + cierre) · metodología iterativa (SDD + BMAD).

Distribución por tipo de actividad

Tipo de actividad	Esfuerzo aprox.
Creación y refinamiento de skills	45 %
Investigación y análisis de código (graphify)	20 %
Validación y feedback	15 %
Documentación y onboarding	15 %
Configuración y setup	5 %

El 65 % del esfuerzo es trabajo técnico (skills + análisis de código); el 35 % restante es validación, documentación y setup.

Roadmap de skills por sprint (incremental)

Los 13 skills se trabajan de forma incremental por sprint, para equiparar el tiempo: Sprint 1 = pipeline (3) + 2 skills; Sprint 2 = +5; Sprint 3 = +3 (se completan los 13). Sprint 4 queda para validación, sin skills nuevos.

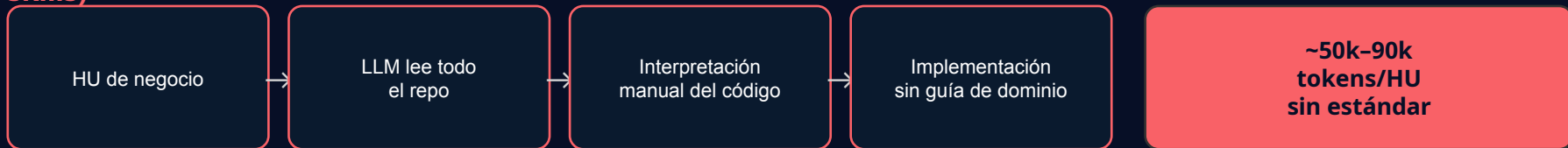
Sprint	Período	Skills que se trabajan (acumulado)	Total
Sprint 1	13–26 ago	Pipeline: bcv-hu-context-analyzer, bcv-dhu-writer, bcv-hu-implementer + bcv-hexagonal-architecture, bcv-java-spring-boot	5 / 13
Sprint 2	27 ago–09 sep	+ bcv-clean-architecture, bcv-openapi-design, bcv-openfeign, bcv-azure-service-bus, bcv-spring-data-jpa-sql-server	10 / 13
Sprint 3	10–23 sep	+ bcv-cosmos-db, bcv-commons-observability, bcv-testing	13 / 13
Sprint 4	24 sep–07 oct	— (validación con equipo dev + estabilización; sin skills nuevos)	13 / 13

Racional: los skills del pipeline se usan en cada HU y son la base; se trabajan primero con 2 skills de creación de código. Luego se suman contratos/integración/persistencia y, al final, observabilidad y testing.

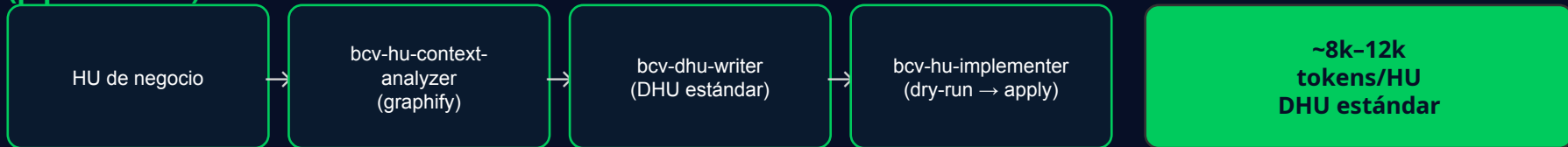
Flujo normal vs. flujo con skills del pipeline

Mismo punto de partida (una HU) · resultado medido con Fernando Camargo en la Semana 2

FLUJO NORMAL (sin skills)



FLUJO CON SKILLS (pipeline BCV)



Medido con Fernando Camargo (equipo dev del banco), Semana 2: ~2 h con la metodología vs. ~8 h (1 día) de la forma tradicional — mismo tipo de tarea.

Roadmap por Sprint

 Completada  En curso  Propuesto



Skills IA · Ecosistema BCV/BACC · 4 sprints de 2 semanas + cierre (13 ago – 09 oct 2026)

SPRINT 1		SPRINT 2		SPRINT 3		SPRINT 4		CIERRE
13–26 ago · 5 / 13 skills		27 ago–09 sep · 10 / 13 skills		10–23 sep · 13 / 13 skills		24 sep–07 oct · validación		08–09 oct · traspaso
SEM 1	SEM 2	SEM 3	SEM 4	SEM 5	SEM 6	SEM 7	SEM 8	SEM C
13–19 ago	20–26 ago	27 ago–02 sep	03–09 sep	10–16 sep	17–23 sep	24–30 sep	01–07 oct	08–09 oct
Consolidación de repos + entorno	Adopción de graphify + creación bcv-hexagonal-architecture y bcv-java-spring-boot	Creación bcv-clean-architecture, bcv-openapi-design y bcv-openfeign	Creación bcv-azure-service-bus y bcv-spring-data-jpa-sql-server	Creación bcv-cosmos-db y bcv-commons-observability	Creación bcv-testing (se completan los 13 skills)	Validación con equipo dev del banco	Estabilización final de skills	Cierre final de entregables
Arranque skills del pipeline (context-analyzer, dhu-writer, implementer)	Ejecución de HU con equipo dev (primer ejercicio real)	Ejecución EVT con equipo dev (context-analyzer → DHU → código)	Consolidar feedback e iterar skills con código real	Refinar skills según feedback	Generar DHU técnica (bcv-dhu-writer)	Ciclo: ajustar → regenerar → revalidar	Documentación, onboarding y demo al cliente	Traspaso a cargo de Oscar

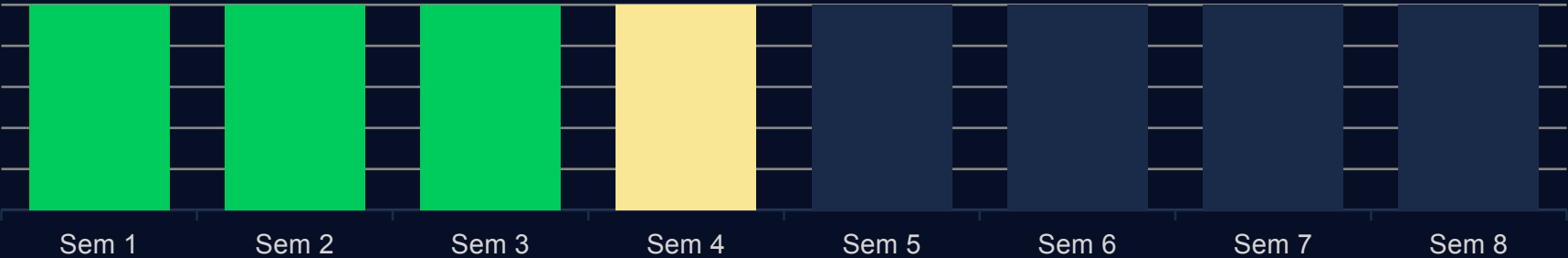
Avance del Roadmap

Semanas numeradas (1-8) del plan de adopción · Actualizado al 08 sep 2026 (Sprint 2 · Semana 4 en curso)

3 de 8 semanas completadas



Estado semana a semana



Hitos clave

Hito	Fecha ref.	Descripción
 Kick-off	13 ago 2026	Inicio oficial del proyecto
 Graphify + skills del pipeline	26 ago 2026	Decisión de ahorro de tokens + creación de los 3 skills del pipeline
 Primera prueba con equipo dev (EVT)	01 sep 2026	Flujo completo validado con Lionel Gonzales — ahorro de ~4 días de implementación
 Culminación Arquitecto IA	10 sep 2026	Traspaso de Jose Luis; Oscar queda como soporte / afinaciones
 Validación con equipo dev del banco	30 sep 2026	Feedback incorporado a los skills
 Cierre del proyecto	09 oct 2026	Skills estabilizados + documentación y onboarding

Dependencias y condiciones de inicio

Dependencia	Responsable	Estado	Impacto si no se resuelve
Entrega de la HU de negocio por el cliente	Interbank	Completada (S2)	Sin HU no se valida el pipeline end-to-end
Entorno de trabajo del equipo dev (PC)	Equipo dev banco	Disponible	Bloquea la validación con feedback
Grafos graphify de los 7 repos	Equipo IA	Generados (S1)	Sin grafos, el análisis consume tokens
Acceso a repositorios BACC	Interbank	OK	Bloquea análisis e implementación
Continuidad tras el 10 sep	Equipo IA	Planificado	Oscar asume soporte y afinaciones

Riesgos del proyecto

Sev.	Riesgo	Mitigación
Alto	Salida de Jose Luis (Arquitecto IA) el 10 sep, a mitad del proyecto	Traspaso planificado; trabajo culminado antes del 10 sep; Oscar cubre soporte y afinaciones
Medio	Dependencia de una HU real del cliente para validar el pipeline	Acuerdo de entrega de HU ya conversado con el cliente
Medio	Feedback del equipo dev del banco fuera de tiempo	El equipo dev ya dispone del entorno en su PC; canal de iteración definido
Medio	service-point-service legacy (Java 17, estructura plana)	Skill bcv-clean-architecture para migrar; no bloquea el resto
Bajo	Grafos graphify desactualizados	Regeneración en CI / check-update

Problemas y riesgos técnicos identificados

Severidad	Problema técnico
Medio	Arquitectura heterogénea: service-point-service es el único sin patrón hexagonal
Medio	rest-api.yaml vacíos en varios servicios
Bajo	Quality gates (SpotBugs / PMD) en skip=true en varios repos
Bajo	Feature flags implementados como propiedades Spring simples, sin plataforma dedicada

Ninguno de estos problemas bloquea el pipeline HU → DHU → código; se gestionan como deuda técnica del ecosistema.

05 · Criterios de Éxito (KPIs)

Cómo mediremos que el proyecto cumplió

Criterios de éxito (KPIs)

Criterio	Indicador
13 skills operativos	Pipeline + implementación validados sobre código real
Ahorro de tokens demostrado	~85 % vs. LLM puro (~8 000–12 000 tokens/HU)
Pipeline validado end-to-end	HU de negocio → DHU → código en ambiente de desarrollo
Estándar corporativo	DHU generada con template ibk-hu-technical-refinement
Feedback incorporado	Correcciones del equipo dev aplicadas en las iteraciones
Onboarding entregado	Documentación + equipo dev autónomo con los skills

Próximos pasos inmediatos

Pasos 1–3 ya ejecutados en el Sprint 2 (01 sep) con Lionel Gonzales sobre un EVT — flujo validado end-to-end con ~4 días ahorrados.

Hecho**Probar el pipeline sobre un EVT**

Validación con Lionel Gonzales (Sprint 2).

Hecho**Ejecutar bcv-hu-context-analyzer**

Análisis con graphify → .context/hu-<codigo>.md.

Hecho**Iterar los skills con código real**

Feedback de la prueba EVT incorporado.

S2**Consolidar el feedback**

Incorporar correcciones de desarrollo (semana 4).

S2**Preparar el traspaso (10 sep)**

Cerrar arquitectura, validaciones y acuerdos con el cliente.

Skills IA para el Ecosistema BCV/BACC

Pipeline HU → DHU → código · 13 skills · ~85 % menos tokens

Próximo hito: validación con el equipo dev del banco y traspaso del Arquitecto IA
(10 sep 2026)